دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

پروژه معماری کامپیوتر
مهندسی کامپیوتر

محاسبه‌ی Reuse Distance

نگارش

شایان بقایی نژاد
آرمین خسروی
رادین شاه‌دائی
باربد شهرآبادی

استاد راهنما

آقای امیرحسین مرادی کهکی

آبان ۱۴۰۱

فهرست مطالب

۱ الگوریتم پیاده‌سازی‌شده ۴

۲ پیاده‌سازی الگوریتم ۴
۱.۲ ساخت Node برای درخت ۴
۲.۲ افزودن Node به درخت ۵
۳.۲ حذف Node از درخت ۶
۴.۲ ساختار کلی درخت ۶
۱.۴.۲ فرآیند Insert در درخت ۷
۲.۴.۲ برقراری توازن درخت ۷
۵.۲ پیدا کردن Reuse Distance در یک آدرس تکراری ۹

۳ پیچیدگی زمان الگوریتم ۱۰

۴ نتایج به دست آمده ۱۰

۵ تحلیل داده‌های مربوط به Reuse Distance ۱۱

فهرست تصاویر

۱ نمودار قرارگیری در Bucket ها ۱۱

۱ الگوریتم پیاده‌سازی‌شده

در مقاله‌ای که در اختیار ما قرار داده شد، به دو روش اشاره شده است که یکی از آن‌ها مبتنی بر پشته (Stack-Based) و دیگری مبتنی بر درخت (Tree-Based) است. روش مبتنی بر پشته به این صورت عمل می‌کند که آدرس‌های مورد دسترسی را در یک پشته ذخیره می‌کند و با دیدن دوباره‌ی یک آدرس (که پیش از این در پشته push شده بود)، یکی یکی خانه‌های درون پشته را pop می‌کند تا به نخستین دسترسی به آدرس تکرار شده برسد. سپس این آدرس را حذف کرده و باقی آدرس‌ها که به طور موقت pop شده‌اند را دوباره درون پشته push می‌کند و هم‌زمان به ازای هر pop موقت، یک واحد Reuse Distance را افزایش می‌دهد. جهت رسیدن به سرعت مورد نیاز و پیدا کردن Reuse Distance در $O(n.log(n))$ نیاز است از روش Tree-Based استفاده کنیم. هم‌چنین برای به دست آوردن سرعت بهتر هنگام اجرای الگوریتم، از زبان C++ برای پیاده‌سازی الگورتیم استفاده کرده‌ایم.
به طور کلی هر Node درخت مورد استفاده در این مسئله تعدادی مقدار را درون خود ذخیره می‌کند. در ادامه به شکل دقیق‌تری به نحوه‌ی ساخت این درخت، ویژگی‌های هر Node و نحوه‌ی استفاده از این درخت برای پیدا کردن Reuse Distance می‌پردازیم.

۲ پیاده‌سازی الگوریتم

۱.۲ ساخت Node برای درخت

برای سادگی یک کلاس می‌سازیم که نماینده‌ی هر Node در درخت است. ساختار این کلاس به شکل زیر است.

```cpp
// Node class represents a single node in the tree
class Node {
public:
    long long content;      // Content of the node
    Node* left;             // Pointer to the left child
    Node* right;            // Pointer to the right child
    Node* parent;           // Pointer to the parent node
    long long height;       // Height of the node in the tree
    long long subtreeSize;  // Size of the subtree rooted at this node

    // Constructor to initialize a new node with given content
    Node(long long content) {
        this->content = content;
        this->left = NULL;
        this->right = NULL;
        this->parent = NULL;
        this->height = 1;
        this->subtreeSize = 1;
    }
};
```

اکنون به توضیحات مربوط به هر بخش در این کلاس می‌پردازیم.

- بخش content: اشاره به اندیس آخرین باری دارد که یک offset در حافظه و در دنباله دیده شده است. هم‌چنین در ادامه خواهیم دید که اندیس آخرین باری که یک offset در دنباله دیده شده است ، به صورت جداگانه در یک Unordered Map نگه‌داری می‌شود.
- بخش left: اشاره‌کننده به Node فرزند سمت چپ نسبت به Node کنونی.
- بخش right: اشاره‌کننده به Node فرزند سمت راست نسبت به Node کنونی.
- بخش parent: اشاره‌کننده به پدر Node کنونی.
- بخش height: مقدار ارتفاع زیردرختی را نگه می‌دارد که Node کنونی ریشه‌ی آن است.
- بخش subtreeSize: تعداد Node های زیردرختی را نگه می‌دارد که Node کنونی ریشه‌ی آن است.

همان گونه که گفته شد، یک Unordered Map نیز از اندیس‌ها نگه می‌داریم:

```
std::unordered_map<std::string, long long> lastOccurrence; // Map to
    store last occurrences
```

۲.۲ افزودن Node به درخت

همان طور که اشاره شد، هر Node اندیس آخرین بار دیده شدن Offset در حافظه را نگه می‌دارد. اکنون با دیدن یک Offset در دنباله، دو حالت می‌تواند رخ دهد.
حالت نخست این است که Offset دیده شده در lastOccurrence Map نیست. در این حالت Node را در درخت Insert می‌کنیم (فرآیند Insert در درخت در ادامه توضیح داده می‌شود) و سپس Map lastOccurence را بروزرسانی می‌کنیم.
در حالتی که Offset در lastOccurrence Map موجود باشد، به این معنی است که پیش از این نیز این Offset دیده شده و در نتیجه باید Reuse Distance آن را محاسبه کنیم که از طریق Rank این Node در درخت قابل انجام است. پس از محاسبه‌ی Reuse Distance ، مقدار آن را به عنوان ورودی به تابع Update Reuse Distance می‌دهیم که در پیچیدگی لگاریتمی مقادیر دریافتی را در تعدادی Bucket ذخیره می‌کند. تقسیم‌بندی Bucket ها به این شکل است:

$$۱,\ ۲,\ ۴,\ ۸,\ ۱۶,\ ۳۲,\ ۶۴,\ ۱۲۸,\ ۲۵۶,\ ۵۱۲,\ ۱/۰k,\ ۲/۰k,\ ۴/۰k,\ ۸/۰k,\ ۱۶/۰k,\ ۳۲/۰k,\ ۶۴/۰k,\ ۱۲۸/۰k,\ ۲۵۶/۰k,\ ۵۱۲/۰k,\ >۵۱۲/۰k$$

پس از این مرحله نیز آن Node را از درخت حذف می‌کنیم و دوباره Insert می‌کنیم. فرآیندهای جدید مانند پیدا کردن Rank یک Node در ادامه مورد بررسی قرار خواهند گرفت.
تکه کد مربوط به این عملیات در شکل زیر قابل مشاهده است.

```
// If the offset was seen before, calculate reuse distance
if (lastOccurrence.find(offset) != lastOccurrence.end()) {
   reuseCount++;
   long long lastIndex = lastOccurrence[offset];
```

```
    long long rank = tree.getRank(lastIndex);
    updateReuseDistance(rank, reuseDistances);
    tree.deleteNode(lastIndex);
}

// Update the last occurrence and insert the current index into the tree
lastOccurrence[offset] = index;
tree.insert(index);
index++;
```

۳.۲ حذف Node از درخت

برای حذف یک Node نیز سه حالت مختلف داریم. حالت نخست زمانی است که Node مورد نظر برگ است. دو حالت دیگر نیز به ترتیب برای زمانی هستند که Node مورد نظر به ترتیب یا فقط یک بچه دارد و یا دو بچه دارد. برای هر کدام از این حالت‌ها توابعی تعریف کرده‌ایم که در فایل مربوط به برنامه موجود می‌باشند.

```
// Internal method to handle node deletion
void deleteNodeInternal(Node* node) {
    if (node->left == NULL && node->right == NULL) {
        deleteLeafNode(node);
    } else if (node->left == NULL || node->right == NULL) {
        deleteSingleChildNode(node);
    } else {
        deleteTwoChildrenNode(node);
    }

    updateSizeAndHeight(node->parent);
    balanceTree(node->parent);
}
```

۴.۲ ساختار کلی درخت

ویژگی حائز اهمیت درخت ساخته‌شده برای این مسئله این است که پیمایش Inorder آن باید ترتیب دیده شدن آدرس‌ها در دنباله را به درستی برگرداند. برای رسیدن به این شرط، به سادگی می‌توان نتیجه گرفت که با دیدن هر آدرس غیرتکراری که در درخت وجود ندارد و یا هر شرایط دیگر (از جمله زمان محاسبه‌ی Reuse Distance)که منجر به Insert کردن در درخت می‌شود، Node جدید باید چپ‌ترین Node درون درخت باشد تا در پیمایش Inorder اولین بار دیده شود.

۱.۴.۲ فرآیند Insert در درخت

برای افزودن یک Node به درخت، غیر از حالتی که درخت خالی است و هیچ ایNode در آن نیست، باید Node را همان طور که گفته شد در سمت چپ‌ترین بخش درخت وارد کرد.

```
// Insert a new node with given content into the tree
void insert(long long content) {
    Node* newNode = new Node(content);
    nodeMap[content] = newNode;
    if (root == NULL) {
        root = newNode;
    } else {
        insertLeftMost(newNode);
    }
    updateSizeAndHeight(newNode);
    balanceTree(newNode);
}

// Insert a node as the leftmost node in the tree
void insertLeftMost(Node* node) {
    Node* current = root;
    while (current->left != NULL) {
        current = current->left;
    }
    current->left = node;
    node->parent = current;
}
```

در شکل بالا دو تابع دیگر نیز دیده می‌شود. یکی از آن‌ها updateSizeAndHeight() است که به سادگی پارامترهای ذخیره شده در Node ها را بروزرسانی می‌کند. تابع دیگر نیز تابع balanceTree() است که در ادامه به آن می‌پردازیم.

۲.۴.۲ برقراری توازن درخت

برای این که بتوانیم در $O(log(n))$ عملیات مختلف مورد نیاز جهت محاسبه‌ی Reuse Distance را انجام دهیم، باید درخت متوازن باشد و فاصله‌ی ریشه‌ی آن تا برگ‌های مختلف تفاوت زیادی نداشته باشد. به همین منظور، پس از انجام عملیاتی مانند Insert یا Delete نیاز است که دوباره درخت را به حالت متوازن (Balanced) برگردانیم. به این منظور از Rotation به چپ و یا راست استفاده می‌کنیم.

```
// Balance the tree starting from the given node
void balanceTree(Node* node) {
    while (node != NULL) {
        long long balance = getBalance(node);
        if (balance > 1) {
```

```
            if (getBalance(node->left) < 0) {
                node->left = rotateLeft(node->left);
            }
            node = rotateRight(node);
        } else if (balance < -1) {
            if (getBalance(node->right) > 0) {
                node->right = rotateRight(node->right);
            }
            node = rotateLeft(node);
        }
        updateSizeAndHeight(node);
        node = node->parent;
    }
}

// Get the balance factor of the given node
long long getBalance(Node* node) {
    if (node == NULL) {
        return 0;
    }
    return getHeight(node->left) - getHeight(node->right);
}
```

همان طور که دیده می‌شود از دو تابع متفاوت برای چرخش به چپ یا راست استفاده می‌شود. در نهایت نیز با توجه به تغییر وضعیت Node درون درخت، ممکن است برخی پارامترهای ذخیره شده در آن تغییر کند. به همین منظور در پایان متوازن کردن درخت، بار دیگر تابع updateSizeAndHeight() را روی Node مورد نظر صدا می‌زنیم تا پارامترهای آن به درستی ذخیره شوند. در کنار آن نیز توابع دیگری مانند updateParentAfterRotation() نیز تمام پارامترهای Node های درخت که نیاز به بروزرسانی دارند را مقداردهی می‌کند. نحوه‌ی چرخش در این درخت نیز مشابه چرخش در درخت‌های AVL و یا Red/Black می‌باشد. توابع مورد استفاده برای Rotation را در شکل بعد می‌بینید.

```
// Rotate the subtree left at the given node
Node* rotateLeft(Node* node) {
    Node* newRoot = performLeftRotation(node);
    updateParentAfterRotation(node, newRoot);
    updateSizeAndHeight(node);
    updateSizeAndHeight(newRoot);
    return newRoot;
}

// Perform left rotation on the given node
Node* performLeftRotation(Node* node) {
    Node* newRoot = node->right;
    node->right = newRoot->left;
    if (newRoot->left != NULL) {
        newRoot->left->parent = node;
```

```
    }
    newRoot->left = node;
    return newRoot;
}
```

```
// Rotate the subtree right at the given node
Node* rotateRight(Node* node) {
    Node* newRoot = performRightRotation(node);
    updateParentAfterRotation(node, newRoot);
    updateSizeAndHeight(node);
    updateSizeAndHeight(newRoot);
    return newRoot;
}

// Perform right rotation on the given node
Node* performRightRotation(Node* node) {
    Node* newRoot = node->left;
    node->left = newRoot->right;
    if (newRoot->right != NULL) {
        newRoot->right->parent = node;
    }
    newRoot->right = node;
    return newRoot;
}
```

۵.۲ پیدا کردن Reuse Distance در یک آدرس تکراری

پیش‌تر گفتیم که برای محاسبه‌ی Reuse Distance نیاز به پیدا کردن Rank آن Node حاوی آدرس داریم. دانشی که Rank یک Node به ما می‌دهد معادل تعداد Node هایی است که پس از آن Node به خصوص به درخت مسئله افزوده شده‌اند. می‌دانیم با دیدن آدرس‌ها هر بار یک Node را به درخت می‌افزاییم. بنابراین می‌توان دریافت که Rank در واقع همان Reuse Distance برای آن آدرس است. تابعی که مقدار Rank را به ما می‌دهد در شکل زیر نشان داده شده است.

```
// Get the rank of a node with the given content
long long getRank(long long content) {
    if (nodeMap.find(content) == nodeMap.end()) {
        throw std::runtime_error("");
    }
    Node* node = nodeMap[content];
    long long rank = 0;
    if (node->left != NULL) {
```

```
        rank += getSize(node->left);
    }

    // Traverse up the tree to calculate the rank
    while (node->parent != NULL) {
        if (node == node->parent->right) {
            rank += getSize(node->parent) - getSize(node);
            node = node->parent;
        } else {
            node = node->parent;
        }
    }
    return rank;
}
```

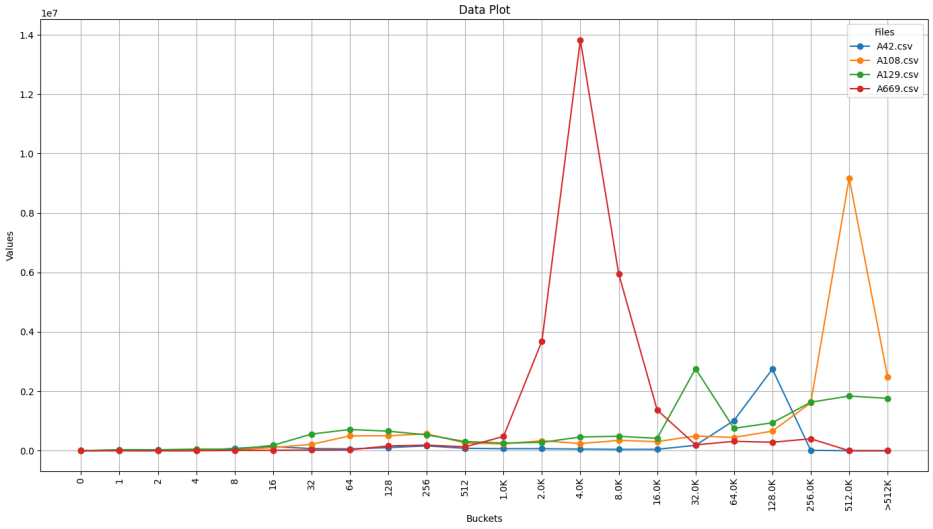
تابع دیگری که از آن استفاده شده، تابع getSize() است که اندازه‌ی زیردرختی را برمی‌گرداند که Node ورودی داده شده به تابع، ریشه‌ی آن است (معادل subTreeSize).
در تابع getRank() از Node ای که قصد مخاسبه‌ی Reuse Distance آن را داریم، آغاز می‌کنیم و در نخستین قدم که هنوز $Rank = 0$ اندازه‌ی زیردرخت چپ Node را به Rank اضافه می‌کنیم. چرا که تمام این Node ها پس از Node مرجع به درخت افزوده شده‌اند. پس از آن از این Node به سمت ریشه حرکت می‌کنیم و هر بار با رسیدن به پدر یک Node ، بررسی می‌کنیم که آیا آن Node که پیش از این در آن بودیم، فرزند چپ پدرش است یا راست. اگر فرزند چپ بود بدون انجام عملیاتی دوباره یک گام دیگر به سمت ریشه می‌رویم. اما اگر فرزند راست بود، به این معنی است که Node های حاضر در زیردرخت سمت چپ پدر، دیرتر به درخت افزوده شده‌اند. پس اندازه‌ی زیردرخت سمت چپ پدر را نیز به Rank می افزاییم. این کار را تا زمانی انجام می‌دهیم که در ریشه باشیم و بخواهیم به پدر آن برویم. چون ریشه پدر ندارد، الگوریتم متوقف می‌شود و Reuse Distance در Rank ذخیره شده است. در نهایت پس از محاسبه‌ی Reuse Distance برای Offset های مختلف، آن‌ها را در Bucket متناظر ذخیره می‌کنیم. در کناز نگه داشتن Reuse Distance ، مقدار بیشینه و کمینه را نیز ذخیره می‌کنیم.

۳ پیچیدگی زمان الگوریتم

دلیل اصلی این که این الگوریتم در $O(nlog(n))$ اجرا می‌شود، متوازن بودن درخت ساخته شده است. به دلیل توازن درخت، هر بار که می‌خواهیم Rank را پیدا کنیم. نیاز است به تعداد ارتفاع درخت مرحله پیش برویم که به دلیل توازن در $O(log(n))$ است. از سوی دیگر عملیات افزودن به درخت یا حذف کردن از درخت نیز در همین زمان قابل انجام است. الگوریتم استفاده‌شده برای حفظ توازن درخت از طریق چرخش نیز این قابلیت را دارد که در تعداد لگاریتمی گام انجام شود.

۴ نتایج به دست آمده

نمودار به دست آمده از قرارگیری هر Reuse Distance در Bucket متناظر، به شکل زیر است.



شکل ۱: نمودار قرارگیری در Bucket ها

مقادیر مربوط به هر فایل csv. نیز به صورت زیر می‌باشد.

File	*avg*	*min*	*max*
A42.csv	*65486*	*0*	*154114*
A108.csv	*324007*	*0*	*1656075*
A129.csv	*235256*	*0*	*2973720*
A669.csv	*7292*	*0*	*178209*

۵ تحلیل داده‌های مربوط به Reuse Distance

از نمودار مشاهده‌شده در بحش پیش می‌توان اطلاعاتی از وضعیت Temporal Locality برنامه‌ها به دست آورد. برای نمونه در برنامه‌ی *A42* می‌توان دید که اکثریت مفادیر Reuse Distance در Bucket با سایز ۴ کیلوبایت قرار می‌گیرد. این موضوع نشان دهنده‌ی این است که به عمده‌ی Locality در فاصله‌ی دسترسی به حدود ۴۰۰۰ آدرس وجود دارد. در نتیجه برای نمونه اگر قصد استفاده از یک حافظه‌ی نهان Fully Associative را داشتیم، در صورتی که اندازه‌ی این حافظه‌ی نهان در پردازنده‌ای Word-Addressable کم‌تر از ۴ کیلوبایت باشد، عمده‌ی Locality مشاهده نشده و Miss Rate در این حافظه‌ی نهان بسیار افزایش می‌یابد. در واقع اگر اندازه‌ی حافظه‌ی نهان کمتر از ۴ کیلوبایت باشد، افزایش اندازه‌ی آن تاثیر آن‌چنانی در عمل‌کرد ندارد.

البته در صورتی که از حافظه‌ی نهان Fully Associative استفاده نکنیم، ممکن است بستگی به Locality Spatial نیاز به حافظه‌ی نهان کوچک‌تری داشته باشیم. همین تجمع در برنامه‌ی *A108* در *512k* رخ می‌دهد. اما در دو برنامه‌ی دیگر روند نسبتاً یکنواخت‌تری مشاهده می‌شود و این یکنواخت بودن باعث می‌شود نقطه‌ی به خصوصی را نتوان برای Cache Size یا Block Size پیدا کرد و در نتیجه نیاز به ببرسی جوانب بیش‌تری باشد.

از سوی دیگر مقادیر غیر صفر در bucket های نه چندان بزرگ و نه چندان کوچک (برای نمونه اندازه‌ی

۸ تا ۵۱۲) نشان می‌دهد که به طور کلی در این برنامه‌ها Spatial Locality نیز وجود دارد و از آن برای بهبود عمل‌کرد دسترسی به حافظه می‌توان استفاده کرد.